

REPORT 01 · FACTORY BENT SIMULATION WITH A BCR ROBOT ARM

Project Overview

Goals, architecture, worlds and how to run the factory bent simulation

ROS 2 Jazzy · Gazebo Harmonic · OpenCV · PyQt5 · BCR 7-axis arm

Hüseyin Tozluyurt · November 2026

1. What the project is

This project is a Gazebo simulation of a small factory cell. Products ride conveyor belts, which the project calls **bents**. A 7-axis **Black Coffee Robotics (BCR) arm** stands between two bents. It uses only what simulated cameras see, processed with **OpenCV**, to decide which product to take and when to take it. It then takes the product to its destination: the second bent, a **recycling bin** or a **trash bin**.

The project began in March 2026 as a single straight conveyor with an overhead camera. By November 2026 it had grown into a complete automation testbed. It has six single-cell worlds, a 13-conveyor factory with 12 arms, 100 realistic product types plus user-made products, a PyQt5 order window, and a vision lab for developing OpenCV and YOLO detectors.



Figure 1. The dual circular factory: bent 1 (left), the BCR arm with the trash and recycling bins, bent 2 (right).

2. Goals

Goal (from the project brief)	Status
Gazebo environments for single, dual circular and arm-integrated bents	Done: 10 world files, generated by scripts
OpenCV object detection on the bents	Done: overhead, side and release cameras, with a cross-check
Pick and place from one bent to another with the BCR arm	Done: the station catch cycle, 6 movement styles, bins
An object generator for the bents	Done: the order GUI and /spawn_order, 100 types, custom 3D products
A development environment for detection algorithms	Done: the vision lab, 10 labs, offline scoring, pipeline tests

3. Architecture

Every part is a separate ROS 2 node. Most of them are thin ROS wrappers around plain Python modules, so the geometry and vision code can be tested without Gazebo.

Node	Role	Talks to
Gazebo + `ros_gz_bridge`	physics, belts (VelocityControl), cameras	camera topics, /joint_states
`ring_object_detector`	overhead OpenCV detection and pixel-to-world conversion	/vision/ring1_objects, /vision/detections
`side_camera_checker`	side view: is a product standing at the station, and how tall is it	/vision/side_check
`release_zone_monitor`	is the drop zone on bent 2 empty	/vision/release_zone
`vision_pick_place`	state machine, URDF IK, movement styles, destinations	joint_trajectory_controller, attacher services
`bcr_endpoint_object_attacher`	grasp and release, hold/pin, belt stop/start, spawning orders, falls into bins	gz-transport set_pose / create / remove
`order_gui`	PyQt5 product orders, destinations and simulation settings	/spawn_order, /factory_settings, bent services

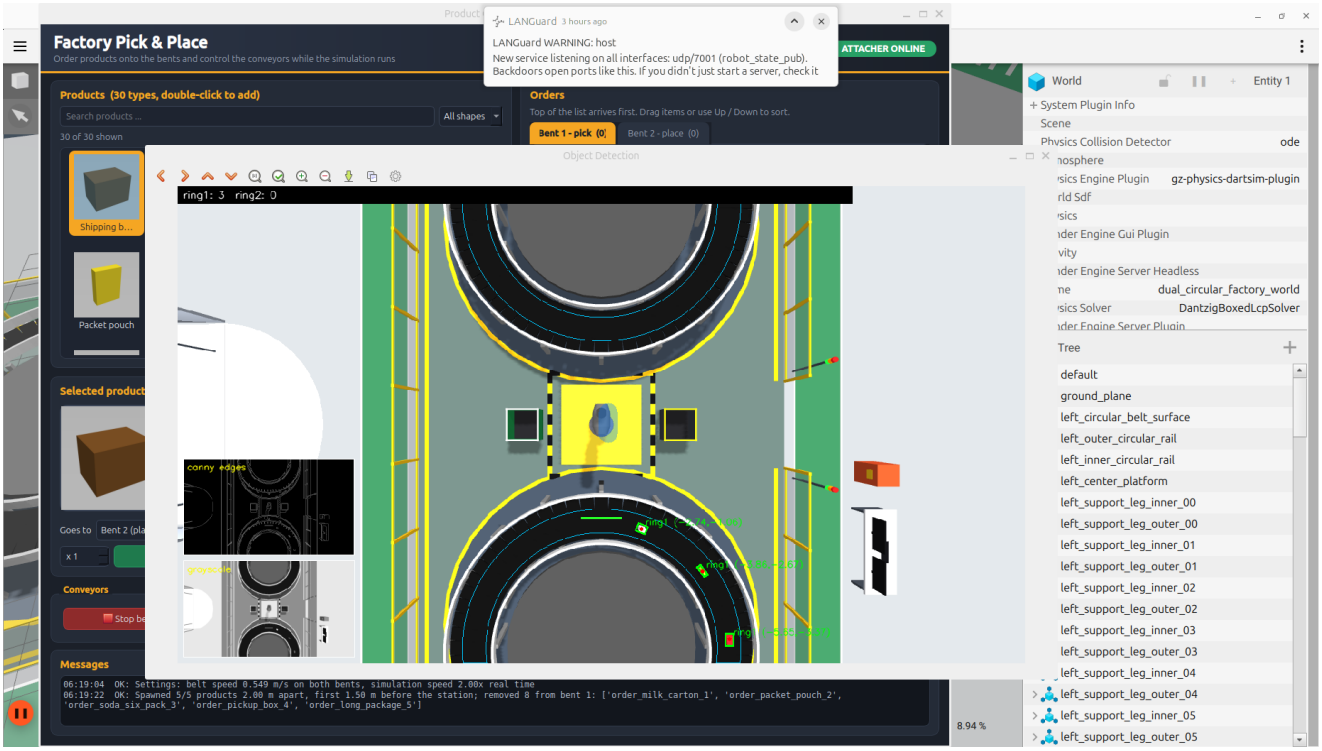


Figure 2. The running system: the order GUI behind, the live Object Detection window in front, Gazebo on the right.

4. One pick-and-place cycle

- The arm waits at its home pose. The overhead camera tracks every product on bent 1.
- A product reaches the catch station at (-2.58, 0). The side camera sees it standing on the belt at the same place (the cross-check).
- The attacher **holds** the product, and bent 1 stops. The arm descends with IK, first quickly and then slowly, until the tip is 1 cm above the product's top.
- The product's top face snaps to `gripper_tip`, and bent 1 runs again. The arm lifts and swings using the chosen movement style.

- For **bent 2**, the arm waits until the release camera reports the zone EMPTY. Then bent 2 stops, the product is placed, and bent 2 runs again. For a **bin**, the arm moves high over the bin centre and drops the product, and the attacher simulates the free fall.
- The arm returns home, and the next product is handled.

5. Worlds

World	Bent 1	Bent 2	Launcher
dual circular	circular	circular	`run_dual_circular.sh`
single circular	circular	–	`run_single_circular.sh`
straight	straight	–	`run_straight_conveyor.sh`
exp straight→circular	straight	circular	`run_exp_straight_to_circular.sh`
exp circular→straight	circular	straight	`run_exp_circular_to_straight.sh`
exp straight→straight	straight	straight	`run_exp_straight_to_straight.sh`
pipeline factory	4 rings, 9 lines	12 arms, 5 docks	`run_pipeline_factory.sh`

6. Running it

```
cd ~/ros2_ws && colcon build --symlink-install && source install/setup.bash
cd src/factory_bent_project
./run_factory.sh                    # order window -> everything starts
./run_factory.sh --world straight-circular motion_type:=cycle
./run_vision_lab.sh sim --world dual  # simulation for the vision lab
./run_vision_lab.sh run 04_sort_by_label
```

The full command reference is `COMMANDS.md` (about 45 KB, 740 lines). The project's history is told in reports 02 to 06.

7. Size of the code base

Part	Size
`factory_bringup/factory_bringup/*.py` (main nodes and modules)	≈ 10,400 lines
`vision_lab/` (framework, detectors, pipeline simulation, tests)	22 modules
World generators (`my_factory_worlds/tools`)	6 scripts
World files	10 SDF files (234 KB to 1.8 MB)
Labs	10 experiment files + 8 templates